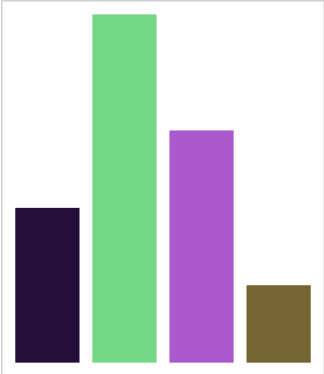
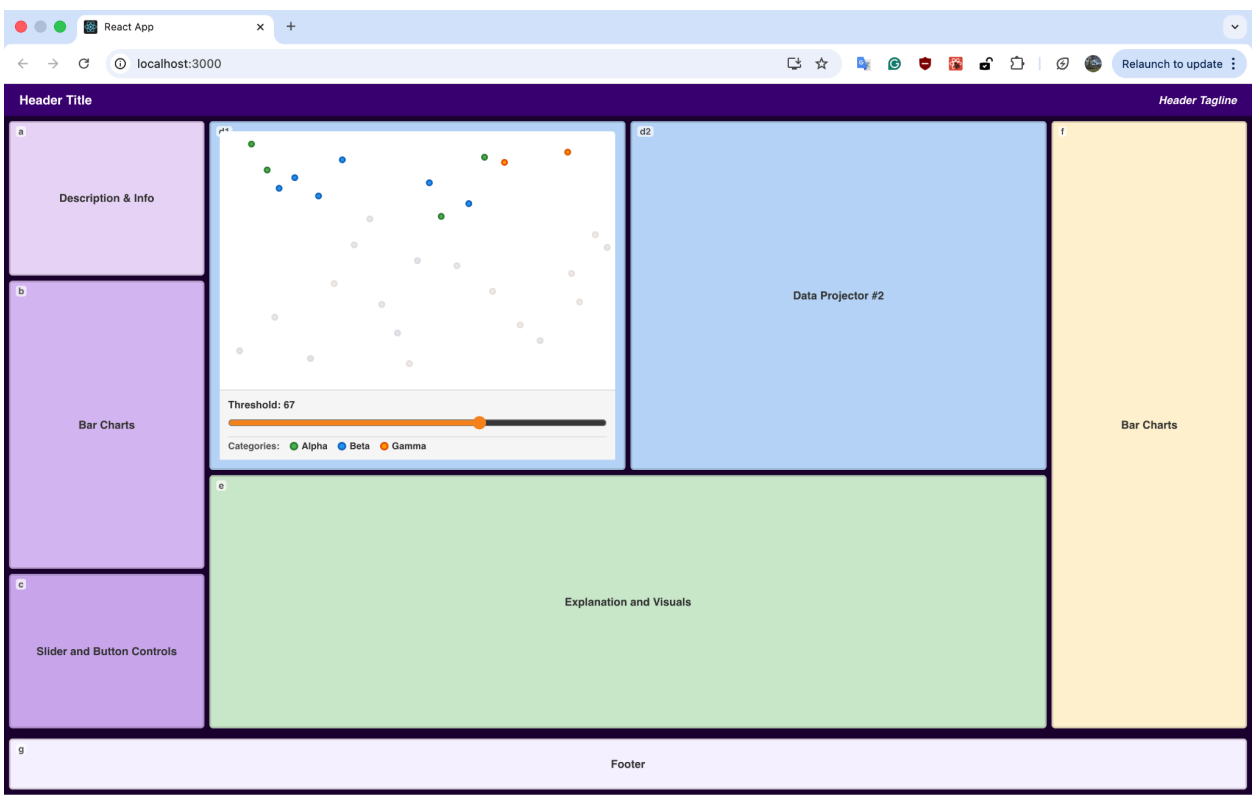
Task 1

```html
<body>
    <div id="chart-container"></div>

    <script>
        const data = [
            { label: 'Server A', value: 40 },
            { label: 'Server B', value: 90 },
            { label: 'Server C', value: 60 },
            { label: 'Server D', value: 20 }
        ];

        const container = document.getElementById('chart-container');

        data.forEach(function (item) {
            const bar = document.createElement('div');
            bar.style.height = item.value * 3 + 'px';
            bar.style.backgroundColor = 'steelblue';
            container.appendChild(bar);

            bar.addEventListener('click', function () {
                const randomColor = '#' + Math.floor(Math.random() * 0xFFFFFF).toString(16).padStart(6, '0');
                bar.style.backgroundColor = randomColor;
            });
        });
    </script>
</body>
</html>
```



Task 2

```
// ScatterPlot Component
function ScatterPlot({ data }) {
  const [threshold, setThreshold] = useState(0);

  return (
    <div className="scatter-plot">
      <div className="scatter-plot-canvas">
        {data.map(point => {
          const isBelowThreshold = point.y < threshold;
          return (
            <div
              key={point.id}
              className={`scatter-point scatter-point-${point.category.toLowerCase()} ${isBelowThreshold ? 'desaturated' : ''}`}
              style={{
                left: `${point.x}%`,
                bottom: `${point.y}%`,
                opacity: isBelowThreshold ? 0.2 : 1
              }}
              title={`(${point.x}, ${point.y}) - ${point.category}`}
            />
          );
        })}
      </div>
      <div className="scatter-controls">
        <div className="control-row">
          <label htmlFor="threshold-slider">
            Threshold: {threshold}
          </label>
          <input
            id="threshold-slider"
            type="range"
            min="0"
            max="100"
            value={threshold}
            onChange={(e) => setThreshold(Number(e.target.value))}
            className="threshold-slider"
          />
        </div>
```



Task 3

Tool Name & Link: https://tailwindcss.com/

Problem Solved: Tailwind CSS eliminates the need to write custom CSS files by providing utility classes for every style property. It solves naming conventions, CSS file organization, and prevents stylesheet bloat through automatic purging of unused styles. The utility-first approach allows rapid prototyping and consistent design systems without leaving your HTML.

Trade-off: Tailwind requires build configuration and learning hundreds of class names. In our project, we wrote semantic CSS classes like `.scatter-plot`, `.legend`, and `.box-projector` which are self-documenting and easy to understand. With Tailwind, our scatter plot div would become " , which is harder to read. For custom layouts like our three-column dashboard, writing traditional CSS gave us precise control and clearer intent.